

课程笔记

CS146S Week 10: What's Next for AI Software Engineering

基于 Martin Casado (a16z) 课程主题整理

2025 年 11 月

课程: Stanford CS146S

学期: Fall 2025

讲次: 1 lecture

网站: <https://themodernsoftware.dev>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 课程回顾：AI 软件开发的全景图	2
1.1 十周课程的知识地图	2
1.2 本章小结	2
2 十年后的软件开发：五大趋势	2
2.1 趋势一：从编码到意图表达	2
2.2 趋势二：软件工程师成为 Agent 团队的管理者	3
2.3 趋势三：开发工具的大整合	3
2.4 趋势四：安全与验证的根本性变革	3
2.5 趋势五：软件的“民主化”与新的分层	3
2.6 本章小结	4
3 不变的核心能力	4
3.1 AI 无法替代的技能	4
3.2 给新一代开发者的建议	4
3.3 未来团队形态的演化	4
3.4 本章小结	5
4 组织层面：保持高速但稳健的过渡	5
4.1 三个阶段的组织准备	5
4.2 组织行动路线图	6
4.3 本章小结	6
5 总结与延伸	6
5.1 课程的核心信息	6
5.2 关键点	6
5.3 个人行动路线	7
5.4 未来岗位分化的可能形态	7
5.5 三阶段迁移路径	7
5.6 拓展阅读	7

1 课程回顾：AI 软件开发的全景图

1.1 十周课程的知识地图

在展望未来之前，让我们回顾这门课所覆盖的 AI 软件开发全景：

1. **Week 1**: LLM 基础与 Prompting 技术——理解 AI 编程工具的底层能力
2. **Week 2**: Coding Agent 与 MCP——构建和标准化 Agent 的工具接口
3. **Week 3**: AI IDE 与 Context Engineering——优化人机协作的上下文环境
4. **Week 4**: Agent Patterns 与 Claude Code——管理多 Agent workflow
5. **Week 5**: Modern Terminal 与产品设计——AI 开发工具的七大产品原则
6. **Week 6**: AI Testing & Security——新型攻击向量与安全防护
7. **Week 7**: AI Code Review——人机协作的代码质量保障
8. **Week 8**: Automated UI/App Building——从自然语言到生产应用
9. **Week 9**: AI DevOps/SRE——生产环境的智能运维
10. **Week 10**: 未来展望——AI 软件工程的下一个十年

从编码到运维的完整生命周期

这门课的独特价值在于它覆盖了软件开发的**完整生命周期**——不仅仅是“用 AI 写代码”，还包括测试、安全、代码审查、应用构建和生产运维。这反映了一个重要现实：AI 正在渗透 SDLC 的每一个环节。

1.2 本章小结

十周课程勾勒出了 AI 软件开发的完整图景，从 LLM 基础能力到生产运维的每个环节都在被 AI 重塑。

2 十年后的软件开发：五大趋势

2.1 趋势一：从编码到意图表达

编程语言的终极演化

如果我们将编程语言的演化视为一条从机器语言到人类语言的轨迹——Assembly → C → Python → 自然语言——那么终极目标是**完全消除语法层面的认知负担**，让开发者只需表达意图 (intent)，AI 负责所有的实现细节。

这不意味着代码会消失。代码仍然是系统运行的底层表示，但人类与代码的交互方式将从“手写”转向“审查和指导”。正如 Boris Cherny (Week 4) 所说：编程语言和 IDE 的生产力都在沿指数曲线增长。

2.2 趋势二：软件工程师成为 Agent 团队的管理者

Week 4 引入的“Agent Manager”概念将在未来十年成为主流。核心转变包括：

- **1 人 = 1 个团队**：一个工程师管理数十个并行运行的 AI Agent
- **技能重心转移**：从编码能力转向规划、委派、架构和审查能力
- **新的工作节奏**：从“写代码-调试-提交”变为“规划-委派-审查-整合”

管理 Agent 和管理人的相似之处

正如 Silas Alberti (Week 3) 指出的：管理 Agent 和管理人一样困难。委派和多线程是需要刻意练习的技能。不善于管理人的工程师同样不会善于管理 Agent。

2.3 趋势三：开发工具的大整合

Week 5 中 Zach Lloyd 提出的问题——代码审查、应用构建、监控等点状解决方案是否会合并为全能平台——很可能在未来十年得到回答。

- **工具碎片化 → 平台整合**：.cursorrules、CLAUDE.md、AGENTS.md 的碎片化将走向统一标准
- **IDE → AI Terminal → AI 操作系统**：开发环境将从代码编辑器演化为全功能的 AI 操作系统
- **MCP 生态成熟**：MCP (Week 2) 作为工具标准化协议将覆盖更多领域

2.4 趋势四：安全与验证的根本性变革

Week 6 揭示的安全挑战——AI Agent 的新型攻击向量、50-100% 的假阳性率——要求安全范式的根本性重构：

- **形式化验证的复兴**：数学证明级别的安全保证将变得更加实用
- **AI 自我对抗**：使用 AI 攻击 AI 生成的代码来发现漏洞 (red-teaming at scale)
- **零信任 Agent 架构**：每个 Agent 的每次操作都需要验证和审计
- **确定性验证层**：在非确定性的 AI 生成之上构建确定性的验证机制

安全是 AI 软件工程的最大未解决挑战

当 AI 编写了大部分代码时，安全的核心问题从“代码是否安全”变为“我们如何信任我们无法完全理解的代码”。这不仅是技术问题，也是组织和法律问题。AI 生成代码的责任归属 (Week 6 的开放问题) 将在未来十年催生新的法律框架。

2.5 趋势五：软件的“民主化”与新的分层

Week 8 展示了 AI 应用构建如何让“anyone can build an app”。这一趋势将加速：

- **更多人可以构建软件**：非技术 PM、设计师、创始人直接构建应用

- **新的分层出现**：“能用 AI 构建 MVP”vs“能构建可扩展的生产系统”之间将产生新的技能分水岭
- **软件数量爆炸**：随着构建成本趋近于零，软件的数量将指数级增长
- **维护成为瓶颈**：构建容易，维护难——大量 AI 生成的应用将面临维护危机

2.6 本章小结

未来十年的五大趋势：意图驱动开发、Agent 团队管理、工具大整合、安全范式重构、软件民主化。每个趋势都有对应的机遇和挑战。

3 不变的核心能力

3.1 AI 无法替代的技能

尽管工具和工作流在快速变化，但 Mihail Eric 在 Week 1 提出的核心理念在未来十年仍然成立：

穿越技术周期的核心能力

1. **业务理解**：理解用户需求和商业价值——AI 不知道“该做什么”
2. **架构思维**：系统级的设计决策需要权衡无数的 trade-off
3. **品味 (Taste)**：区分好代码和坏代码、好设计和坏设计的能力
4. **代码阅读能力**：审查和理解 AI 生成的代码比自己写代码更重要
5. **沟通与协调**：无论是管理人还是管理 Agent，清晰的沟通是基础

3.2 给新一代开发者的建议

基于整门课程的洞察，对未来开发者的几点建议：

1. **学习 AI 工具，但不要依赖它们**：理解底层原理（Week 1-2）比熟练使用某个工具更持久
2. **投资于阅读代码的能力**：在 AI 编码时代，审查能力比编写能力更有价值（Week 7）
3. **练习委派和多线程工作**：这是 Agent Manager 模式的核心技能（Week 3-4）
4. **理解安全的基本原则**：AI 引入了新的攻击面，安全素养是不可协商的（Week 6）
5. **拥抱快速实验**：正如 Mihail 在 Week 1 所说，“没有成熟的范式，每个人都在摸索”
6. **为 6 个月后的模型构建**：Boris Cherny 的建议（Week 4）适用于所有工具和工作流的选择

3.3 未来团队形态的演化

如果把整门课的结论映射到组织层面，未来的软件团队很可能出现新的分层：

团队层次	主要工作方式	竞争优势来源
个人开发者层	借助 AI IDE 和 Agent 完成端到端任务	试错速度、跨职能执行力
小团队层	多人各自管理多个 Agent 并协同集成	规划能力、接口边界和 review 机制
平台团队层	维护统一的工具链、规范、评测与安全基线	组织规模化复制能力

表 1: AI 软件工程时代的团队形态变化

未来的组织优势会更多来自系统设计，而不是单点高手

当个人编码效率被 AI 大幅放大后，真正拉开差距的往往不再是“谁写得更快”，而是“谁能把工具、流程、安全和审查体系组装成一个稳定系统”。这也是整门课最后回到组织与流程视角的原因。

3.4 本章小结

技术变化越快，基础能力越重要。业务理解、架构思维、品味、代码阅读和沟通是穿越 AI 技术周期的永恒技能。

4 组织层面：保持高速但稳健的过渡

4.1 三个阶段的组织准备

想要把 AI 软件工程的愿景落到组织层面，需要同时兼顾速度与稳健，以下是常见的三个阶段：

1. **实验阶段**：小团队尝试 Agent、AI IDE、App Builder；重点是快速验证想法
2. **制度化阶段**：建立命名约定、review guardrails、测试与监控指标；重点是可持续性
3. **系统化阶段**：把 AI 作为产品交付链的一部分；强调安全、审计、组织协作与可评估的反馈

最快的组织演进不在于大规模复制，而在于建立可复用的 guardrails

许多团队在第一个阶段看到“AI 生成的成功原型”，就迫不及待地向第二个阶段跳跃。这经常导致 guardrails 缺失、审查混乱、旧流程被抛弃。真正的演进是先建立可复制的 guardrails，再逐步扩大影响力。

4.2 组织行动路线图

阶段	关键行动	成功信号
实验	试点 Agent + context engineering	快速迭代 + 总结 playbook
制度化	统一 review、测试、监控指标	Guardrails 成为“新常态”
系统化	安全、负责人与治理机制到位	变更跨部门可审计 + 责任明确

表 2: AI 软件工程组织演进路线图

4.3 本章小结

组织层面的演进必须以可复用的 guardrails 为基础，逐步从实验走向系统化。最重要的指标不是工具本身，而是 guardrails 是否被实际遵守、审查是否可回放、责任是否明确。

5 总结与延伸

5.1 课程的核心信息

CS146S 这门课的核心信息可以归结为一句话：**AI 正在改变软件开发的每一个环节，但优秀工程师的核心价值——理解力、判断力和品味——不仅不会被取代，反而变得更加珍贵。**

Martin Casado 作为 a16z 的 General Partner，从投资人的视角为我们提供了一个更宏观的思考框架。他在 VMware 和 Nicira 的创业经历证明：真正的技术革命不是替代人类，而是**放大人类的能力**。AI 软件工程也不例外。

终极展望

十年后的软件开发者将不再“写”代码——他们将**设计、指导、审查和编排**软件的创建。编码将成为一种“低级操作”，由 AI Agent 完成，就像今天的编译器把高级语言转换为机器码一样。但就像编译器没有取代程序员一样，AI Agent 也不会取代软件工程师——它们只是将“软件工程”的含义提升到了一个新的抽象层次。

5.2 关键点

- 五大趋势：意图驱动开发、Agent 管理、工具整合、安全重构、软件民主化
- 核心不变：业务理解、架构思维、品味、代码阅读、沟通能力
- AI 放大人类能力而非替代人类
- “Software engineering” 的含义将提升到新的抽象层次
- 现在是拥抱快速实验的最佳时机

5.3 个人行动路线

1. 在未来 3 个月内建立一套稳定的 AI 编程工作流，而不是只零散试用工具
2. 把测试、安全、review 和文档一起纳入 AI 工作流，而不是只优化写代码速度
3. 主动训练自己做规划、委派和系统审查，这些会比纯编码更快成为稀缺能力

5.4 未来岗位分化的可能形态

如果把整门课的判断再往前推一步，软件行业内部可能出现新的岗位重心分化：

- **AI Workflow Engineer**：专门设计 Agent 工作流、行为文件、工具链和验证层
- **System Reviewer / Architect**：更专注于审查系统边界、接口契约和长期演进方向
- **AI-Native Product Builder**：兼具产品、设计和工程能力，快速完成从需求到原型再到上线

岗位不会简单减少，而是重新组合

AI 可能减少某些低层重复编码工作，但会同时放大规划、审查、集成、安全和组织设计的重要性。对个人而言，真正的风险不是“AI 会不会写代码”，而是自己是否仍停留在只会执行低层任务的角色里。

5.5 三阶段迁移路径

阶段	团队主要变化	对个人的直接要求
短期（1-2 年）	AI IDE / Agent 成为默认工具	学会把测试、review 和文档一起纳入 AI 工作流
中期（3-5 年）	单人管理多 Agent 成为常态	强化规划、委派、系统集成和审查能力
长期（5-10 年）	软件工程抽象层继续上移	以业务理解、架构判断和组织设计作为核心竞争力

表 3: AI 软件工程的三阶段迁移路径

5.6 拓展阅读

- 课程官网：<https://themodernsoftware.dev>
- a16z AI 观点：<https://a16z.com/ai/>
- Martin Casado 的博客与演讲：<https://a16z.com/author/martin-casado/>
- “Software 2.0” by Andrej Karpathy
- “The End of Programming” by Matt Welsh (Communications of the ACM)
- “Why Software Is Eating the World” by Marc Andreessen (a16z)